Lab 5. Simple KVS

System Programming Assignment

Sinu Lee
Security & Reliable Systems Lab

What You Should Do

1. Implement a basic server
   - Use socket APIs
   - Use multiple threads

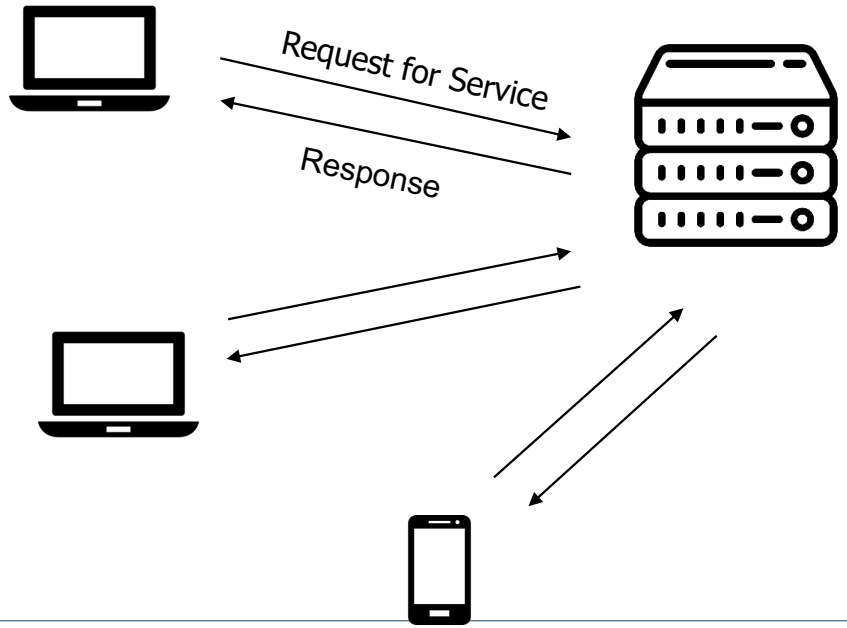
2. Implement a global hash table and rwlock
   - Multiple threads may access at the same time
   - Global hash table uses rwlock library
   - rwlock should support multiple concurrent readers when there is no writer

- Reference for socket programming in C:
  - https://beej.us/guide/bgnet/
- Reference for rwlock:
  - https://docs.oracle.com/cd/E37838_01/html/E61057/sync-124.html
  - https://www.ibm.com/docs/en/aix/7.3?topic=p-pthread-rwlock-rdlock-pthread-rwlock-tryrdlock-subroutines
  - https://docs.kernel.org/locking/locktypes.html

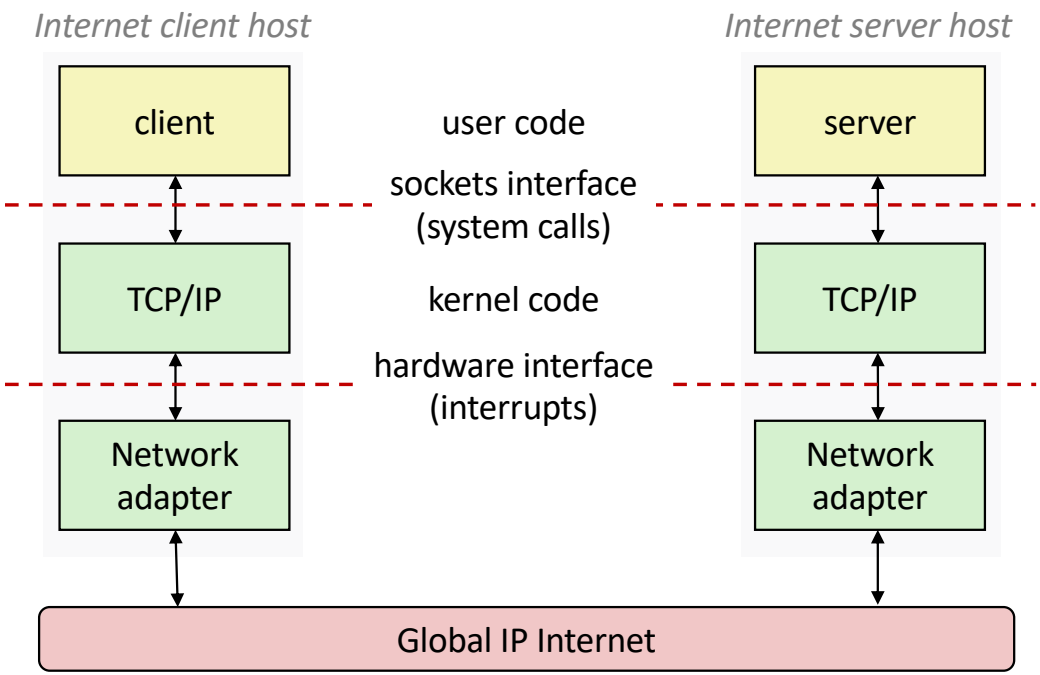
Background 1:
Basic Server/Client

What is the Client-Server Model?

- **Definition**: A distributed computing architecture where a **client** requests services, and a **server** provides them.
- **Structure**: Typically involves multiple clients connecting to a centralized server.

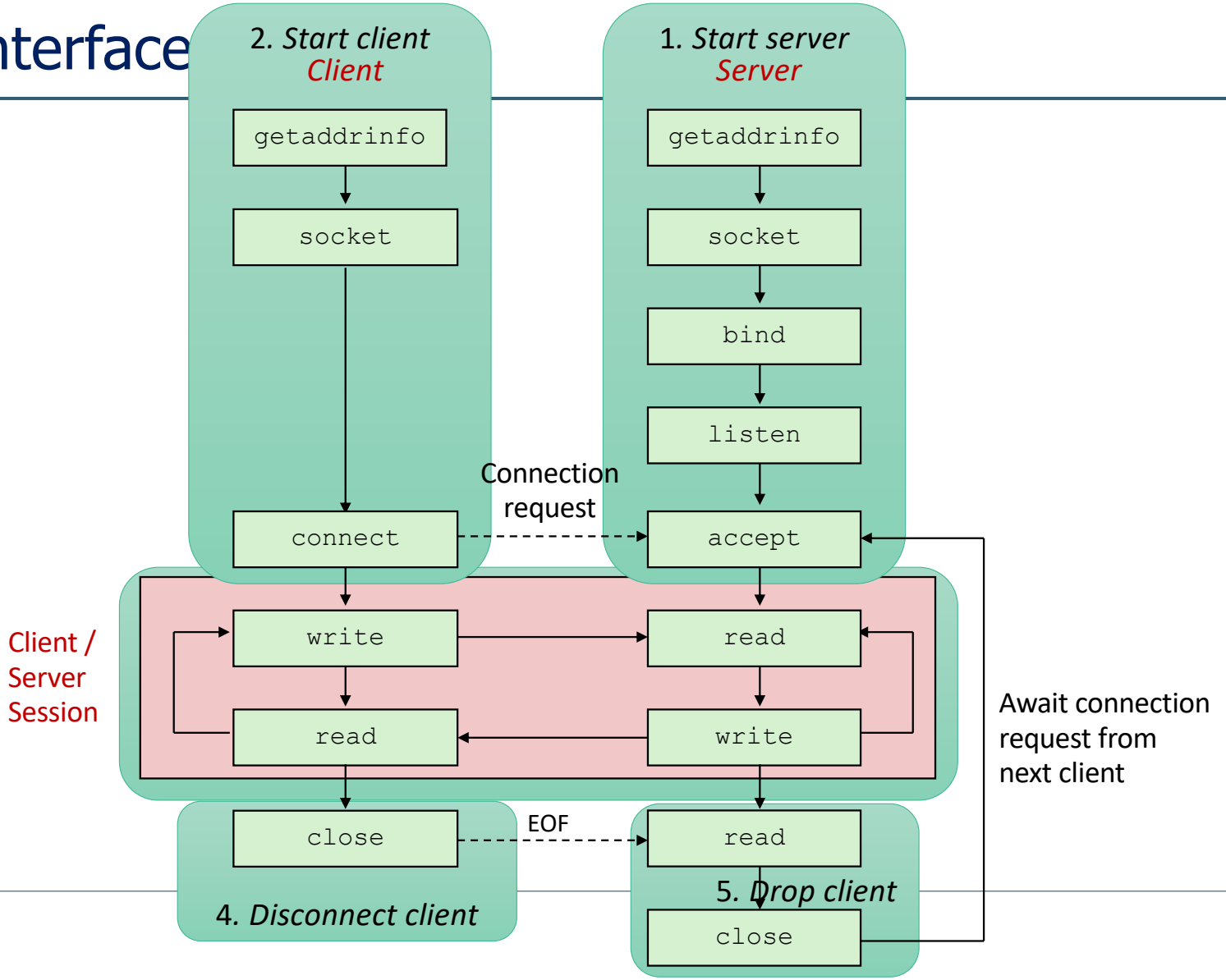


Programmer's View



- Both client and server use socket interface!
- Not only for TCP, but also for UDP

TCP Socket Interface



System Calls for Basic Server

- `socket()`
  - Makes a socket and allocates system resources for the socket
- `bind()`
  - Binds IP address and port to the socket
- `listen()`
  - Creates a connection queue to allow connections from clients
- `accept()`
  - Retrieves a connection from the connection queue and **returns a socket** for an established connection
- `read() & write()`
  - Next slide
- `close()`
  - Close the socket and cleans up resources

Socket descriptor,
Socket structure for metadata

For this assignment, use 0.0.0.0
for binding server IP address

Listen, accept: only for TCP
TCP server has 2 sockets!

System Calls for Basic Client

- `getaddrinfo() & freeaddrinfo()`
  - Helper function for IP lookup
  - You may use `gethostbyname()` or `gethostbyname_r()`
- `socket()`
  - Makes a socket and allocates system resources for the socket
- `bind()`
  - Binds IP address and port to the socket
  - Usually not used in client
- `connect()`
  - Creates a connection to the specified address:port (starts TCP handshake)
- `read() & write()`
  - Next slide
- `close()`
  - Close the socket and cleans up resources

Socket descriptor,
Socket structure for metadata

connect: for TCP
TCP client has 1 socket

System Calls for Socket Options

- `int setsockopt(int sockfd, int level, int optname, const void *optval, socklen_t optlen);`
- `int getsockopt(int sockfd, int level, int optname, void *optval, socklen_t *optlen);`
- For address/port reuse and timeout setting
- `level`: Specifies the protocol layer for the option
  - `SOL_SOCKET` : Socket layer (e.g., `SO_REUSEADDR`) ← use this
  - `IPPROTO_TCP` : TCP layer (e.g., `TCP_NODELAY`)
  - `IPPROTO_IP` : IP layer (e.g., `IP_TTL`)
- `optname`: `SO_REUSEADDR, SO_REUSEPORT, SO_RCVTIMEO`
- `optval, optlen`: different by optname.
  - `SO_REUSEADDR, SO_REUSEPORT` : `int value`, `1` means on / `0` means off
  - `SO_RCVTIMEO` : `struct timeval to`, set `to.tv_sec = TIMEOUT;`
- You can check out its usage in skeleton code!

read()

- `ssize_t read(int fd, void *buf, size_t nbytes)`

- `Returns`
  - # of bytes read (> 0)
  - -1 (error, you should check `errno`)
    - No bytes read
  - 0 (closed by the peer)
- `errno == EAGAIN, EWOULDBLOCK`
  - Nothing to read from the TCP socket recv buffer, try later
- `errno == EINTR`
  - Failed due to interrupt, try again right now
- `errno == ECONNRESET`
  - The peer abruptly reset the connection

write()

- `ssize_t write(int fd, void *buf, size_t nbytes)`

- `Returns`
  - # of bytes wrote (> 0)
  - -1 (error, you should check `errno`)
    - No bytes wrote

- `errno == EAGAIN, EWOULDBLOCK`
  - TCP socket send buffer is full, try later
- `errno == EINTR`
  - Failed due to interrupt, try again right now
- `errno == ECONNRESET`
  - The peer abruptly reset the connection
- `errno == EPIPE`
  - The peer closed the connection (since SIGPIPE kills the process first, you will not see this error)

Background 2: Lock

Spin Lock vs. Mutex Lock

- Spin Lock
  - Continuously checks if the lock is available while consuming CPU cycles (busy-waiting)
  - Optimized for short wait times and multi-core environments
- Mutex Lock
  - Puts the waiting thread to sleep, releasing the CPU until the lock becomes available
  - Suitable for longer wait times or resource-intensive applications

Aspect	Spin Lock	Mutex Lock
Waiting	Busy-waiting (CPU is fully utilized)	Sleep-waiting (CPU is released)
Overhead	Low (no context switching)	High (context switching involved)
Use case	Short wait times	Long wait times
Multi-core	Effective in multi-core systems	Works well in both single/multi-core
Complexity	Simple	Relies on OS support

Mutex Lock

- Protects shared data
- Allows only one thread to access the critical section

```c
int shared_data = 0;
void *thread_function(void* arg) {
    pthread_mutex_lock(&mutex);  // acquire mutex lock

    // critical section: access to the shared data
    shared_data++;
    printf("Thread %ld incremented data: %d\n",
           (long)arg, shared_data);

    pthread_mutex_unlock(&mutex);  // release mutex lock
    return NULL;
}
```

- What if most of the accesses are reads?

Many Readers & Few Writers

- Contention necessary?

```c
int shared_data = 0;
void *reader_function(void* arg) {
    pthread_mutex_lock(&mutex);

    printf("Reader %ld read data: %d\n",
           (long)arg, shared_data);

    pthread_mutex_unlock(&mutex);
    return NULL;
}
```

```c
void *writer_function(void* arg) {
    pthread_mutex_lock(&mutex);

    shared_data++;
    printf("Writer %ld updated data: %d\n",
           (long)arg, shared_data);

    pthread_mutex_unlock(&mutex);
    return NULL;
}
```

- If there is no concurrent writer, multiple threads can concurrently read

RW Lock

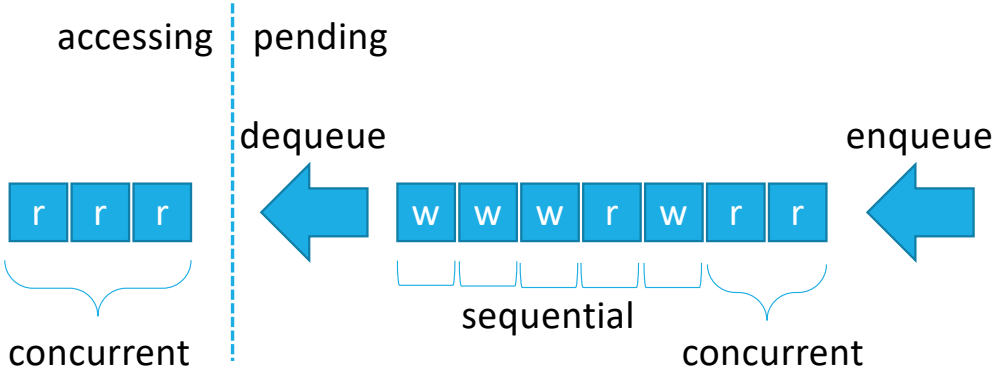
- A specialized lock that distinguishes between read and write access
- RW lock is usually **reader-priority** (pthread_rwlockattr_setkind_np(3) - Linux manual page)

1. Allows concurrent multiple readers
   - Blocks writer when any readers exist (including pending readers)
   - When there is no more readers, wake up the oldest pending writer
2. Allows single writer only
   - Blocks other threads when writing
   - After finishing, wake up all pending readers

Aspect	Mutex Lock	Reader-Writer Lock
Concurrency	Only one thread (reader or writer) at a time	Multiple readers or a single writer
Performance	Suitable for low read-to-write ratio	Optimized for high read-to-write ratio
Complexity	Simpler	More complex
Starvation Risk	No starvation (single queue)	Writer starvation
Use Case	Any critical section	Scenarios with frequent reads and rare writes

FIFO RW Lock

1. Allows concurrent multiple readers
2. Allows single writer only
3. Handle all accesses FIFO



Aspect	Reader-Priority RW Lock	FIFO RW Lock
Concurrency	Multiple readers or a single writer	Multiple readers or a single writer
Performance	Optimized for high read-to-write ratio	Balanced performance and fairness
Starvation Risk	Writer starvation	No starvation
Use Case	Scenarios with frequent reads and rare writes	Any scenarios

https://docs.kernel.org/locking/locktypes.html
ReentrantReadWriteLock (Java Platform SE 8)
https://docs.rs/tokio/latest/tokio/sync/struct.RwLock.html

Signaling through Condition Variables

- Example for waking up other threads using condition variables

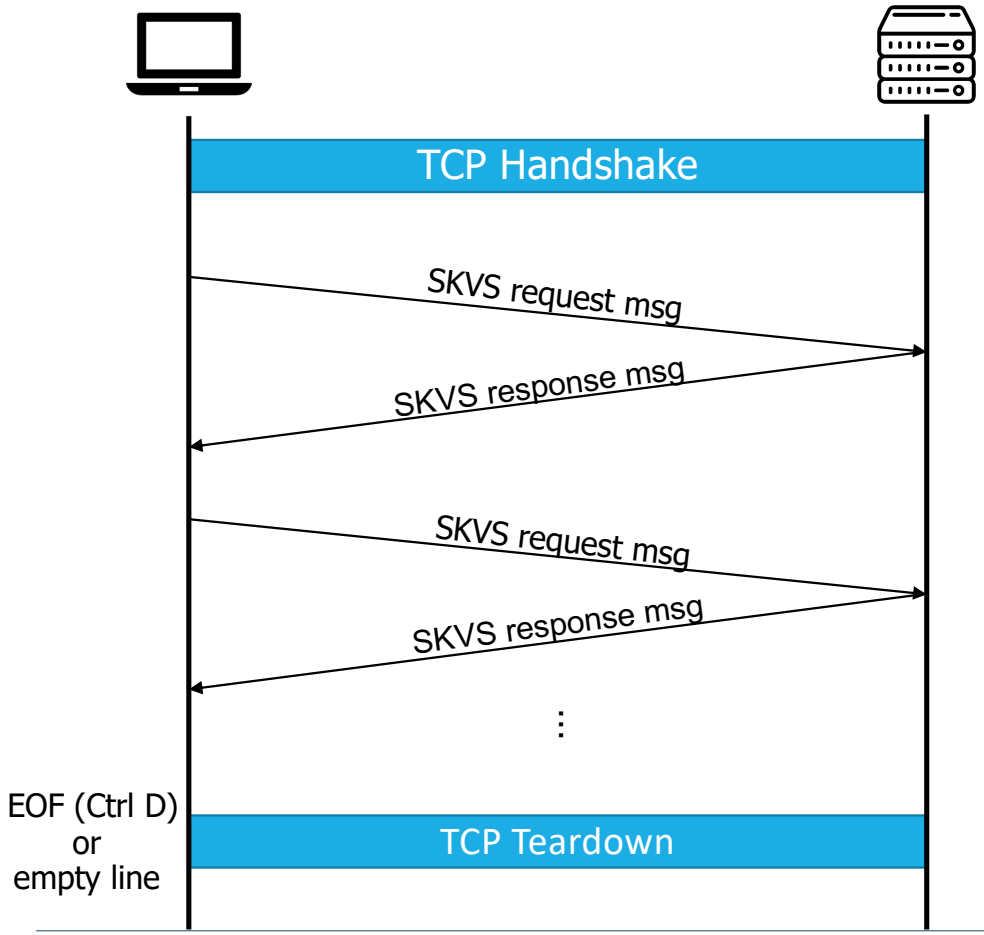
Thread 1: `pthread_cond_wait(&condvar, &mutex);`

Thread 2: `pthread_cond_signal(&condvar);`

`(or pthread_cond_broadcast(&condvar);)`

Part 1:
SKVS Server

SKVS Protocol

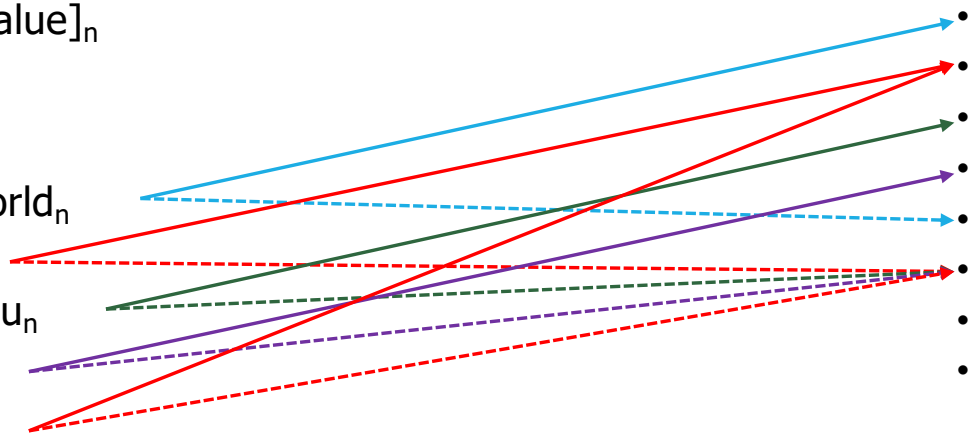


- One connection per one client
- **Keep alive** until typing empty line (\n) or EOF (Ctrl D)
- **Half-duplex**
  - After sending request, wait for the response
  - No need to respond pipelined requests
- **Text** based protocol
  - Key and value should be also text
  - Refer to the next slide
- Server should be **stateful**
  - Key-value pairs should be accessible by other clients
- Default service port: 8080

SKVS Protocol (cont.)

"$_s$": a space character
"$_n$": a newline character

- Request Msg Format:
  1. $[CMD]_s[key]_s[value]_n$
  2. $[CMD]_s[key]_n$
- Examples
  - $CREATE_shello_sworld_n$
  - $READ_shello_n$
  - $UPDATE_shello_ssnu_n$
  - $DELETE_shello_n$
  - $QREAD_shello_n$

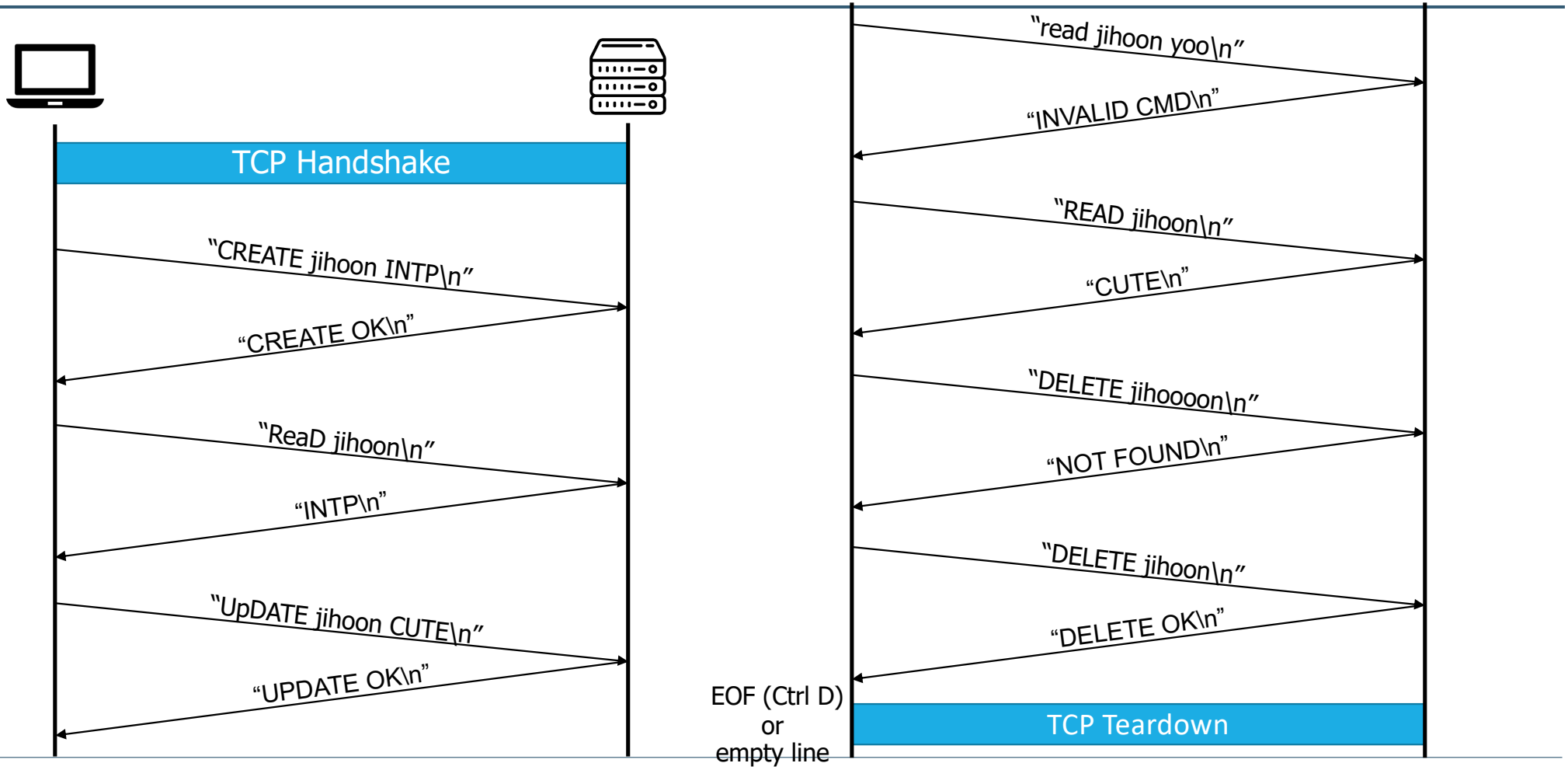


- Response Msg Format:
  - $CREATE_sOK_n$
  - $[value]_n$
  - $UPDATE_sOK_n$
  - $DELETE_sOK_n$
  - $COLLISION_n$
  - NOT_sFOUND_n
  - $INVALID_sCMD_n$
  - $INTERNAL_sERR_n$

- CMD: one of CREATE, READ, UPDATE, DELETE, QREAD
  - case-insensitive (e.g., rEAd, cReAtE are okay)
- Key, value: string without $_s$ nor $_n$
  - No binary, only text, case-sensitive
  - len(key) <= 32B, len(SKVS msg) <= 4096B

- EOF or empty line on client
  - Close the connection
  - Exit client program

Example

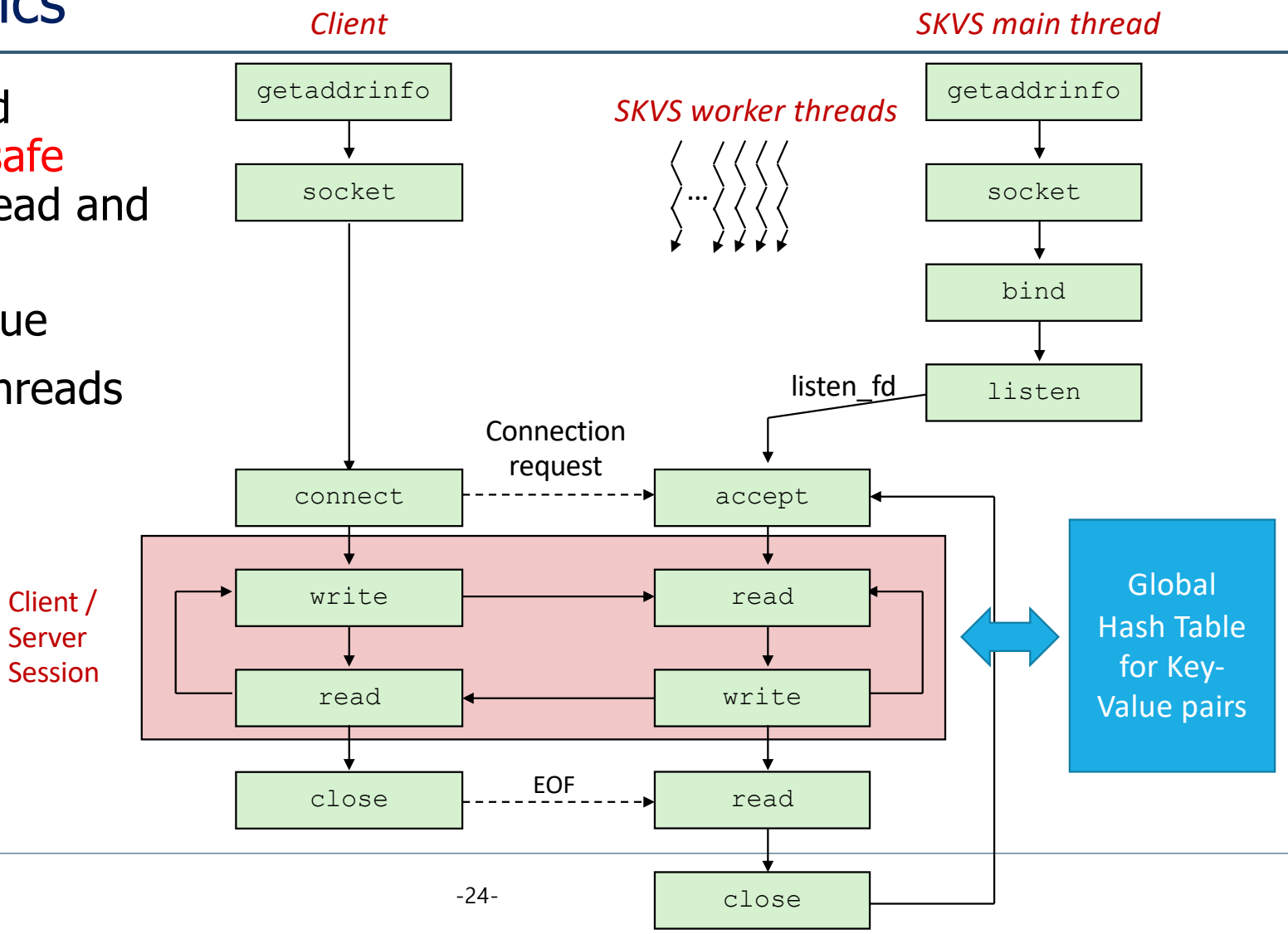

TCP Handshake
"CREATE jihoon INTP\n"
"CREATE OK\n"
"ReaD jihoon\n"
"INTP\n"
"UpDATE jihoon CUTE\n"
"UPDATE OK\n"
"read jihoon yoo\n"
"INVALID CMD\n"
"READ jihoon\n"
"CUTE\n"
"DELETE jihoooon\n"
"NOT FOUND\n"
"DELETE jihoon\n"
"DELETE OK\n"
EOF (Ctrl D)
or
empty line
TCP Teardown

Handling Errors

- If SKVS request is in invalid format?
  - Responds "INVALID CMD\n"
- If the length of SKVS msg > 4096
  - Reference client will prohibit this case
- If the length of [key] > 32
  - Responds "INVALID CMD\n"
- Received CREATE msg, but key-value pair already exists?
  - Responds "COLLISION\n"
- Received READ/UPDATE/DELETE/QREAD msg, but no such key exists?
  - Responds "NOT FOUND\n"
- If any internal error occurs, so cannot serve the request?
  - Responds "INTERNAL ERR\n"

SKVS Semantics

- Global HT should support thread-safe access to both read and write operations
- Single listen queue
- Use 10 worker threads



Why Static 10 Threads? Any Other Ways for Concurrency?

- What you have learned
  - `accept(), read(), and write()` are blocking..
  - What if dynamically `fork() / pthread_create()` for every clients?

  (+) Easy to implement

  (+) Easy to leverage multiple cores

- Reality..

  (--) Wastes system resources

  (--) Poor performance due to creating processes or threads

  (--) Poor performance due to context switching

  - Using `fork() / pthread_create()` for each request is inefficient (heavyweight)

Event-Driven Socket Programming (Out of Scope for this time)

- Set the sockets non-blocking
- Use event-driven API (e.g., `epoll`)
- Static threads to leverage multiple cores
- Then,
  - (+) Best performance
    - Supports concurrency without context switching
    - Small number of threads
  - (--) Hard to implement using event-driven APIs

➔ Workaround: Simply use static threads for this assignment! ☺

SKVS API

- Helper API for parsing and processing the SKVS requests
- `struct skvs_ctx *skvs_init(size_t hash_size, int delay)`
  - Initiates SKVS context
- `void skvs_destroy(struct skvs_ctx *ctx, int dump)`
  - Destroys SKVS context
- `int skvs_serve(struct skvs_ctx *ctx, char *rbuf, size_t rlen, char *wbuf, size_t *wlen)`
  - Writes a complete response to wbuf for the given SKVS request in rbuf
  - e.g., [value]\n, "INVALID CMD\n", "CREATE OK\n", "UPDATE OK\n", "COLLISION\n", …

Server Requirement

- Your server should serve ~10 concurrent clients
- Your server should interoperate with reference client

- Reference server/client binaries will be provided
  - No support for Mac, windows (but, will work on WSL)
- For usage, refer to the reference server/client

Usage Example

- On the client, SKVS responses will be printed out to stdout

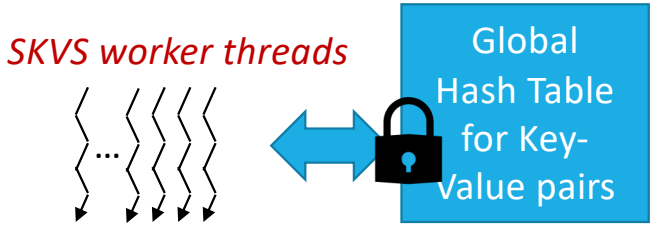
```
junghan@sp01:~/lab-5-simple-kvs/assign5/ref$ ./server -p 8000 -t 5 -s 4096
Server listening on 0.0.0.0:8000
1th worker ready
0th worker ready
3th worker ready
2th worker ready
4th worker ready
```

```
junghan@sp02:~/lab-5-simple-kvs/assign5/ref$ cat input
create hello world
read hello
junghan@sp02:~/lab-5-simple-kvs/assign5/ref$ ./client -i sp01.snucse.org -p 8000 < input
CREATE OK
world
```

Part 2:
RW Lock

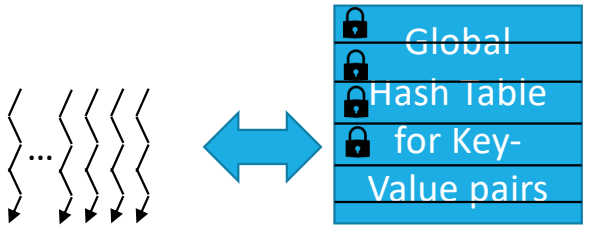
Lock

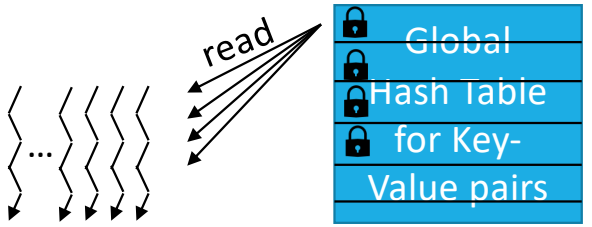
- SKVS uses a global hash table
  - Any threads consistently access the key-value pair
  - Accesses to the hash table needs lock



- But, lock contention leads to poor performance…

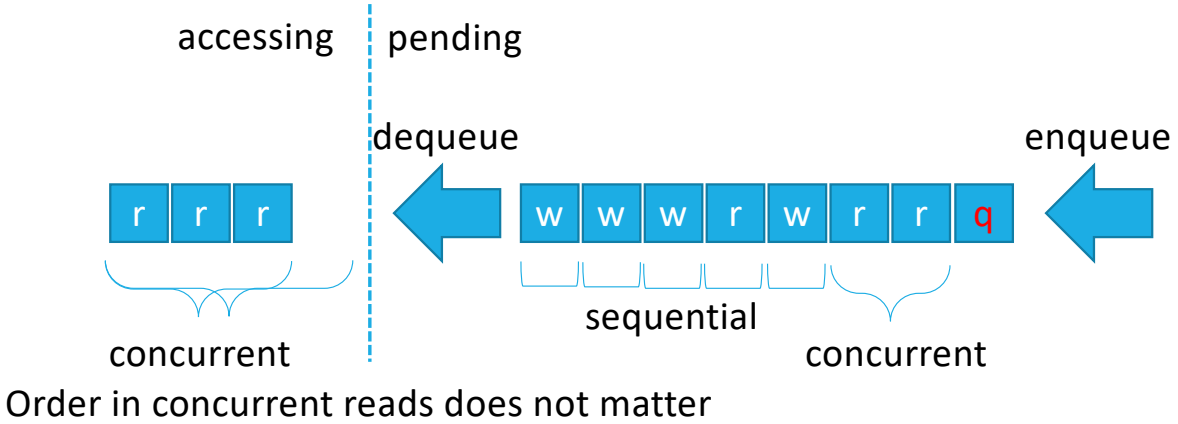
1. Fine-grained lock
   - **For each bucket**
   - Less contention compared to course-grained lock
2. Custom RW lock
   - Assumes many readers, few writers
   - **Multiple readers are allowed at the same time**
   - By default, read and write are handled in **FIFO order**
   - But, **Quick read preempts** pending accesses





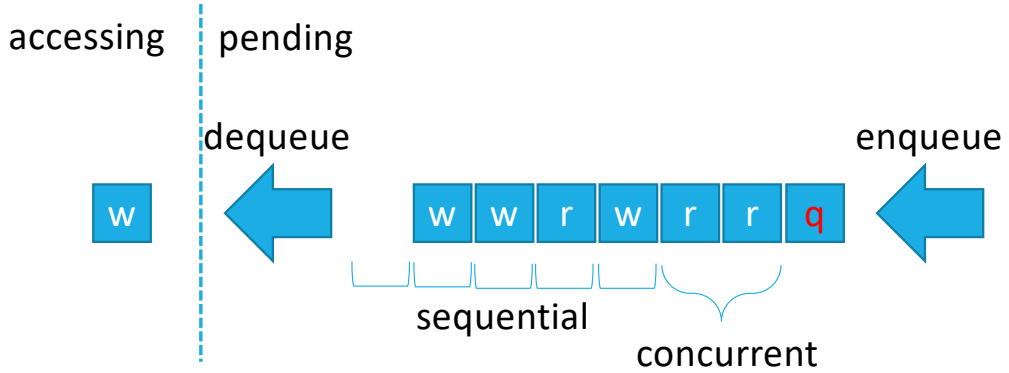
Custom RW Lock: FIFO RW Lock + Quick read

- Handle requests in FIFO by default
- Readers can be concurrently handled
- Writers should be sequentially handled
- "Quick read" preempts pending accesses (prioritized read)
- "Quick read" enqueued while processing concurrent read?



Custom RW Lock: FIFO RW Lock + Quick read (2)

- Handle requests in FIFO by default
- Readers can be concurrently handled
- Writers should be sequentially handled
- "Quick read" preempts pending accesses (prioritized read)
- "Quick read" enqueued while processing sequential write?



rwlock.c

- Combination of FIFO RW lock and Reader-priority RW lock
- **Not** allowed to use `pthread_rwlock` API
- Implement your **own** one in `rwlock.c` using `pthread_mutex` and `pthread_cond` API

- `int rwlock_init(rwlock_t *rw, int delay)`
- `int rwlock_read_lock(rwlock_t *rw, int quick)`
- `int rwlock_read_unlock(rwlock_t *rw)`
- `int rwlock_write_lock(rwlock_t *rw)`
- `int rwlock_write_unlock(rwlock_t *rw)`
- `int rwlock_destroy(rwlock_t *rw)`

hashtable.c

- `int hash_insert(hashtable_t *table, char *key, char *value)`
  - Needs a **write lock**
  - Duplicates key and value
  - Fills node structure and inserts node to the table
  - Returns 0 if collision, 1 if inserted, -1 if any internal error
- `int hash_read(hashtable_t *table, char *key, char *dst, int quick)`
  - Needs a **read lock**
  - Returns 0 if not found, 1 (and outputs to `value`) if found, -1 if any internal error
  - If found, **copies the value to dst before releasing rlock**
  - **If quick set, should preempt pending accesses**

hashtable.c

- `int hash_update(hashtable_t *table, char *key, char *value)`
  - Needs a **write lock**
  - Duplicates value
  - Updates node->value to new one
  - Free old value
  - Returns 0 if not found, 1 if updated, -1 if any internal error
- `int hash_delete(hashtable_t *table, char *key)`
  - Needs a **write lock**
  - Frees key and value
  - Evicts from the table
  - Returns 0 if not found, 1 if deleted, -1 if any internal error

Dumping Global Hash Table

- Your SKVS server should dump the hash table using `skvs_destroy(ctx, 1)` before exit on SIGINT
- Register SIGINT handler for dump

Requirements

- Concurrent multiple readers' access to the global hash table
- Correct RW lock semantic (FIFO + reader-priority)
- Print the entries in the global hash table using `hash_dump()`
  - Do not modify this function

Debugging and Testing

Reference Binaries for self-checking

- Reference server/client binaries will be provided
  - No support for Mac, windows
- For any ambiguities, please refer to the reference server/client

- You may use `TRACE_PRINT()` and `DEBUG_PRINT()` for debugging

```
CFLAGS += -DTRACE
CFLAGS += -DDEBUG
```

Self-testing for Write Lock

- Before testing, check below
  1. Implement server/client first with **robust read/write**
  2. Your server **must** be ready for concurrency first
  3. `./server -d 5`
     `sleep(5)` in `rwlock_write_unlock()`, `rwlock_read_unlock()`

- Example test scenario

1. Send "CREATE hello 1\n" on client 1
   - It will hold a write lock for 5 seconds
2. Send "CREATE hello 2\n" on client 2
   - Check whether client 2 gets stuck
3. Send "CREATE hello 3\n" on client 3
   - Check whether client 3 gets stuck
4. Send "CREATE hello 4\n" on client 4
   - Check whether client 4 gets stuck

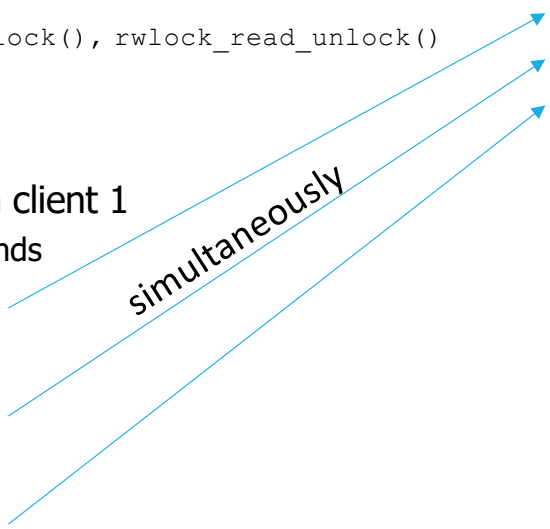
7. After 5 seconds…
   - Check whether client 1 receives "CREATE OK\n"
8. After 5 seconds…
   - Check whether client 2 receives "COLLISION\n"
9. After 5 seconds…
   - Check whether client 3 receives "COLLISION\n"
10. After 5 seconds…
    - Check whether client 4 receives "COLLISION\n"

Self-testing for Read Lock

- Before testing, check below
  1. Implement server/client first with **robust read/write**
  2. Your server **must** be ready for concurrency first
  3. `./server -d 5`
     `sleep(5)` in `rwlock_write_unlock()`, `rwlock_read_unlock()`

- Example test scenario

1. Send "CREATE hello world\n" on client 1
   - It will hold a write lock for 5 seconds
2. Send "READ hello\n" on client 2
   - Check whether client 2 gets stuck
3. Send "READ hello\n" on client 3
   - Check whether client 3 gets stuck
4. Send "READ hello\n" on client 4
   - Check whether client 4 gets stuck



7. After 5 seconds…
   - Check whether client 1 receives "CREATE OK\n"
8. After 5 seconds…
   - Check whether client 2 receives "world\n"
   - Check whether client 3 receives "world\n"
   - Check whether client 4 receives "world\n"

Self-testing for FIFO Scenario

- Before testing, check below
  1. Implement server/client first with **robust read/write**
  2. Your server **must** be ready for concurrency first
  3. `./server -d 5`
     `sleep(5)` in `rwlock_write_unlock()`, `rwlock_read_unlock()`
- Example test scenario

1. Send "CREATE hello world\n" on client 1
   - It will hold a write lock for 5 seconds
2. Send "DELETE bye\n" on client 2
   - Check whether client 2 gets stuck
3. Send "READ hello\n" on client 3
   - Check whether client 3 gets stuck
4. Send "UPDATE hello snu\n" on client 4
   - Check whether client 4 gets stuck
5. Send "DELETE hello\n" on client 5
   - Check whether client 5 gets stuck
6. Send "READ hello\n" on client 6
   - Check whether client 6 gets stuck

7. After 5 seconds…
   - Check whether client 1 receives "CREATE OK\n"
   - Check whether client 2 receives "NOT FOUND\n" (why?)
8. After 5 seconds…
   - Check whether client 3 receives "world\n"
9. After 5 seconds…
   - Check whether client 4 receives "UPDATE OK\n"
10. After 5 seconds…
    - Check whether client 5 receives "DELETE OK\n"
11. After 5 seconds…
    - Check whether client 6 receives "NOT FOUND\n"

Self-testing for Quick Read Scenario

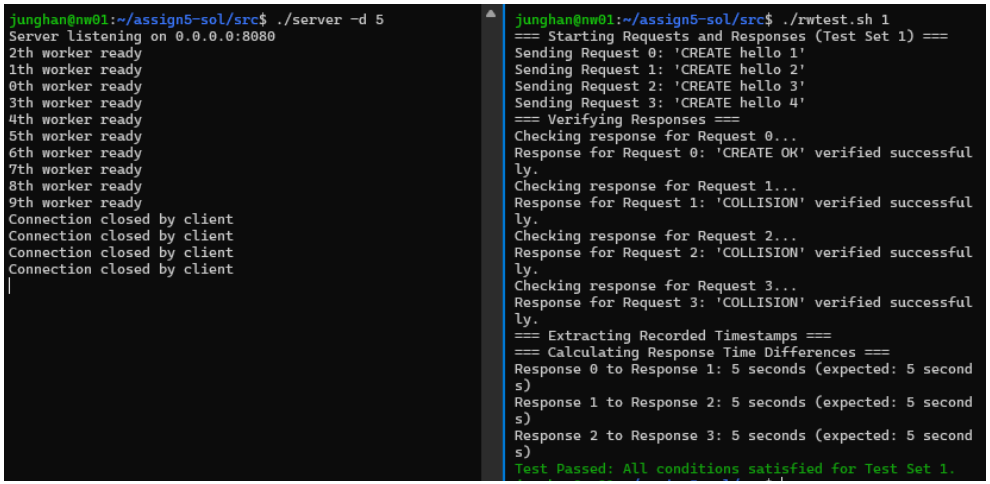
- Before testing, check below
  1. Implement server/client first with **robust read/write**
  2. Your server **must** be ready for concurrency first
  3. `./server -d 5`
     `sleep(5)` in `rwlock_write_unlock()`, `rwlock_read_unlock()`
- Example test scenario

1. Send "CREATE hello world\n" on client 1
   - It will hold a write lock for 5 seconds
2. Send "DELETE bye\n" on client 2
   - Check whether client 2 gets stuck
3. Send "READ hello\n" on client 3
   - Check whether client 3 gets stuck
4. Send "UPDATE hello snu\n" on client 4
   - Check whether client 4 gets stuck
5. Send "DELETE hello\n" on client 5
   - Check whether client 5 gets stuck
6. Send "QREAD hello\n" on client 6
   - Check whether client 6 gets stuck

7. After 5 seconds…
   - Check whether client 1 receives "CREATE OK\n"
   - Check whether client 2 receives "NOT FOUND\n" (why?)
8. After 5 seconds…
   - Check whether client 3 receives "world\n"
   - Check whether client 6 receives "world\n"
9. After 5 seconds…
   - Check whether client 4 receives "UPDATE OK\n"
10. After 5 seconds…
    - Check whether client 5 receives "DELETE OK\n"

Automated Self-testing

- Run your server with delay 5 first, and run `rwtest.sh` on another terminal



- There are 5 default tests, but you can extend them for more complex test cases
- To run `rwtest.sh`, you need to install timestamp by "`sudo apt install moreutils`"

Guidelines

Notice

- Do not change the name and the prototype of the skeleton code
- You are allowed to modify server.c, hashtable.c, and rwlock.c
  - Do not modify header files, skvslib.c
- Carefully read slides, readme, and header files.

- What to submit
  - A tarball named `202612345_assign5` including `server.c hashtable.c rwlock.c`

```
cd assign5/src
make submit ID=202612345
```

- Replace `202612345` to your student ID without dash

Deadline

- Deadline: ~ 2026. 06. 16 23:59
  - 0 Points if deadline is missed
  - 0 Points for copying
- Contact
  - Please use ETL for normal Q&A.
  - TA mailing list: sp-2026-spring-ta@googlegroups.com
  - If you need to contact TA via Email, please attach [LAB 5] as a prefix of title.

Criteria

- build (10)
- single create test (5)
- single read test (5)
- single update test (5)
- single delete test (5)
- concurrency test (10)
- write lock test (10)
- read lock test (10)
- fine-grained lock test (10)
- complex scenario test (15)
- complex scenario test including quick read (15)